

HydroSense Kenya

A Climate-Aware Scientific Computing System

From Numerical Methods to Decision
Intelligence

ICS 2207 Capstone Project

Prepared by **Caleb Mutai**

The Scientific Problem (The 'Why')



Context

Kenyan smallholder farms face erratic rainfall and acute water scarcity.



Central Scientific Question:

'How can we model water availability and simulate future soil moisture to recommend an irrigation plan that minimizes water use without exposing crops to stress?'



Goal:

Transitioning from 'Guess-based' traditional farming to 'Data-driven' decision support systems.

The Mathematical Model

⚡ Core Water Balance:

$$S_{(t+1)} = S_t + R_t + I_t - ET_t - D_t$$

⚡ ET: Simplified empirical formula:

$$et = 0.12 * T + 0.35 * W + 2.4 * Solar - 0.025 * H$$

⚡ Interpretation: Water balance is a dynamic, continuous physical process that requires daily high-frequency tracking of atmospheric variables.

Data Audit & Cleaning (Level 4)

- **Findings:** Identified logically invalid temperature spikes ($>45^{\circ}\text{C}$) and missing rainfall records (NA).
- **Scientific Decision:** Raw sensor data cannot be trusted blindly. Skepticism led to the aggregation of sub-daily noise into stable daily means.
- **Action:** Applied Forward-Filling for soil moisture and Capping for temperature outliers at 38°C .

`</>` Aggregating Weather daily data

```
def aggregate_daily_weather(df: pd.DataFrame) -> pd.DataFrame:
    # Ensure timestamp column exists
    if 'ts' not in df.columns:
        raise KeyError("Expected column 'ts' not found in DataFrame")

    # Convert to datetime
    df['timestamp'] = pd.to_datetime(df['ts'])

    # Group by date and aggregate
    daily = (
        df.groupby(df['timestamp'].dt.date)
        .agg({
            'rg1': 'sum',          # total rainfall
            'temp_sht': 'mean',    # average temperature
            'humidity_sht': 'mean',
            'wind_spd': 'mean',
            'si1145_uv': 'max'     # peak solar intensity
        })
        .reset_index()
    )

    # Rename columns for clarity
    daily.columns = [
        'date', 'rainfall_mm', 'temp_c',
        'humidity_pct', 'wind_speed_mps', 'solar_index'
    ]

    return daily
```

Numerical Engine Validation (Level 3)

-  **Technique:** Manual implementation of Bisection, Newton-Raphson, and Gaussian Elimination.
-  **Audit Defense:** All manual algorithms validated against NumPy/SciPy reference results with an error margin $< 10^{-7}$.
-  **Interpretation:** Precision in root-finding is the boundary between under-watering (crop stress) and over-watering (resource waste).

Simulation Stability: Euler vs. RK4 (Level 5)



Comparison

The Euler method 'drifts' over a 30-day period due to accumulation of truncation errors.



Precision

4th-order Runge-Kutta (RK4) provides a mathematically stable solution for long-term planning.



Conclusion

Higher-order methods are essential to maintain numerical integrity in climate-aware forecasting.



Runge-Kutta Method

```
def rk4_step(S, R, I, ET, coeff, fc, dt=1):  
    """4th Order Runge-Kutta Step for higher precision."""  
    k1 = dt * moisture_rate_of_change(S, R, I, ET, coeff, fc)  
    k2 = dt * moisture_rate_of_change(S + 0.5*k1, R, I, ET, coeff, fc)  
    k3 = dt * moisture_rate_of_change(S + 0.5*k2, R, I, ET, coeff, fc)  
    k4 = dt * moisture_rate_of_change(S + k3, R, I, ET, coeff, fc)  
    return S + (k1 + 2*k2 + 2*k3 + k4) / 6
```



Euler Method

```
def euler_method(s_start, days, params):  
    """Simulates soil moisture over time using Euler approximation"""  
    results = [s_start]  
    for t in range(days):  
        # Simplified change rate  
        ds_dt = params['rainfall'][t] - params['et'][t]  
        results.append(results[-1] + ds_dt)  
    return results
```

Dealing with Uncertainty: Monte Carlo (Level 5)

Strategy

Modeled 1,000 unique futures using stochastic rainfall noise (Normal Distribution).

Key Statistic

'Quantified the risk of Zone B hitting the wilting point without intervention.'

Conclusion

Scientific computing allows us to move from 'average' predictions to probabilistic risk assessments.

Optimization: Gradient Descent (Level 5)

- The Cost Function: $J(l) = (\text{Moisture Error})^2 + \text{Lambda} * (\text{Water Use})^2$
- The Method: Iterative descent toward the 'Sweet Spot' of minimum water use for maximum crop health.
- Interpretation: Achieved resource efficiency by balancing biological needs against operational constraints.

Reproducibility & AI Ethics (Level 6)

-  **Evidence:** 11 automated tests covering every mandatory numerical requirement.
-  **AI Responsibility:** AI used as a multiplier for test generation and debugging, while maintaining manual accountability for physics/math logic.
-  **Audit Ready:** Structured repository following scientific standards (data/src/tests/notebooks).

Final Scientific Conclusions

- **Modeling Works:** Simple water-balance equations, when solved with high-order methods (RK4), provide a reliable roadmap for smallholder farmers.
- **Stochastic Advantage:** Probabilistic simulations are superior to static models for climate-aware planning.
- **Scalability:** This modular system can be adapted for any crop zone (Tomato, Kale, Maize) across East Africa.